



Università dell' Aquila

ROOC: linguaggio ed ambiente integrato per la
modellazione e soluzione di problemi di ottimiz-
zazione lineare

Department of Information Engineering, Computer Science and Mathematics
Corso di Laurea in Computer Science

Candidato

Enrico Menichelli

Matricola 279061

Relatore

Prof. Matteo Spezialetti

Correlatore

Prof. Stefano Smriglio

Anno Accademico 2025/2026

ROOC: linguaggio ed ambiente integrato per la modellazione e soluzione di problemi di ottimizzazione lineare

Tesi di Laurea. Università dell' Aquila

© 2025 Enrico Menichelli. Tutti i diritti riservati

Questa tesi è stata composta con L^AT_EX e la classe uaqthesis.

Email dell'autore: specy.dev@gmail.com

Contents

1	Introduzione	1
1.1	Contesto e motivazione	1
1.2	Il problema e l'idea di ROOC	1
1.3	Obiettivi e contributi	2
1.4	Struttura della tesi	2
2	Background e stato dell'arte	3
2.1	Programmazione lineare e MILP	3
2.2	I modelli di ottimizzazione	3
2.3	Linguaggi e strumenti di modellazione esistenti	4
2.4	Cenni di teoria dei linguaggi e dei compilatori	4
2.5	Posizionamento di ROOC	4
3	Il linguaggio ROOC	5
3.1	Principi di design	5
3.2	Struttura di un modello	5
3.3	Esempio guida	5
3.4	Primitive e tipi di dato	6
3.5	Iteratori, <i>block functions</i> e destructuring	6
3.6	Operatori e definizione delle variabili	6
3.7	Libreria standard e funzioni definite dall'utente	6
4	Architettura del compilatore	7
4.1	La pipeline multi-fase	7

4.2	L'astrazione <i>Pipe</i>	7
4.3	Estensibilità e visualizzazione passo-passo	8
5	Analisi sintattica e rappresentazione intermedia	9
5.1	Concetti teorici	9
5.2	Applicazione in ROOC: la grammatica PEST	9
5.3	Il PreModel e l'Intermediate Language	10
5.4	Gestione e tracciamento degli errori	10
6	Analisi semantica: tipi e trasformazione del modello	11
6.1	Concetti teorici	11
6.2	Applicazione in ROOC: il sistema dei tipi	11
6.3	Il type checker	12
6.4	Trasformazione PreModel $\rightarrow$ Model	12
7	Linearizzazione e forma standard	13
7.1	Concetti teorici	13
7.2	Applicazione in ROOC: da Model a LinearModel	13
7.3	Linearizzazione di min, max e valore assoluto	14
7.4	Trasformazione in forma standard	14
8	Risoluzione dei modelli	15
8.1	Concetti teorici	15
8.2	Applicazione in ROOC: il semplice a due fasi	15
8.3	microlp: fork ed estensione a MILP	16
8.4	Backend esterni e <i>auto-solver</i>	16
8.5	Confronto qualitativo	16
9	Dalla libreria alla piattaforma web	17
9.1	Compilazione in WebAssembly e la libreria TypeScript	17
9.2	L'IDE web e l'LSP nel browser	17
9.3	Valore didattico	18

10 Conclusioni e sviluppi futuri	19
10.1 Riepilogo dei contributi	19
10.2 Limitazioni	19
10.3 Sviluppi futuri	19
Appendices	20
A Grammatica completa di ROOC	20
B Esempi di modelli	21
Bibliography	22

Chapter 1

Introduzione

Capitolo introduttivo: presentare il contesto, il problema, gli obiettivi e i contributi della tesi. Stile discorsivo, una pagina o due.

1.1 Contesto e motivazione

I problemi di ottimizzazione sono pervasivi in ambito industriale, logistico e scientifico, ma la loro formalizzazione in un modello matematico risolubile richiede competenze specialistiche e strumenti spesso poco accessibili. In questo capitolo si introduce il contesto della *Ricerca Operativa* e dell'*Ottimizzazione Combinatoria* da cui nasce il progetto.

Descrivere brevemente perché modellare problemi di ottimizzazione è difficile e perché serve un linguaggio dedicato.

1.2 Il problema e l'idea di ROOC

Introdurre ROOC: un linguaggio di modellazione formale che, a partire da un modello e dai relativi dati, produce un modello lineare risolubile con tecniche di ottimizzazione. Anticipare l'idea della pipeline di compilazione.

1.3 Obiettivi e contributi

Elencare in modo puntuale i contributi: (1) il design del linguaggio, (2) il compilatore multi-fase, (3) il solver MILP `microlp`, (4) la libreria riusabile (Rust/WASM/TypeScript) e la piattaforma web didattica.

1.4 Struttura della tesi

Riassumere il contenuto di ciascun capitolo successivo.

Chapter 2

Background e stato dell'arte

Capitolo di preambolo. Fornisce le basi teoriche minime (programmazione lineare e MILP), inquadra i linguaggi di modellazione esistenti e introduce i concetti di teoria dei compilatori che verranno applicati nei capitoli successivi.

2.1 Programmazione lineare e MILP

Si richiamano i concetti fondamentali della programmazione lineare (LP): la funzione obiettivo, i vincoli lineari, le variabili decisionali e la nozione di soluzione ottima. Si introduce poi la programmazione lineare intera mista (MILP), in cui alcune variabili sono vincolate a essere intere o binarie.

Definire formalmente un modello LP/MILP (forma generale). Distinguere variabili continue, intere e binarie. Citare la forma standard, che verrà usata nel Capitolo 7.

2.2 I modelli di ottimizzazione

Spiegare in modo intuitivo cosa sia un modello di ottimizzazione, con un piccolo esempio (es. knapsack o dominating set), che farà da filo conduttore.

2.3 Linguaggi e strumenti di modellazione esistenti

Esistono numerosi linguaggi e ambienti per la modellazione di problemi di ottimizzazione. Se ne analizzano i principali, evidenziandone caratteristiche e limiti, per posizionare ROOC nel panorama esistente.

Confrontare: AMPL, GNU MathProg (GMPL/GLPK), MiniZinc, Gurobi/OPL, Pyomo, JuMP, e il formato testuale CPLEX LP. Per ognuno: paradigma, espressività, accessibilità, licenza, supporto a strutture dati (grafi, tuple).

2.4 Cenni di teoria dei linguaggi e dei compilatori

ROOC è, a tutti gli effetti, un compilatore che traduce un linguaggio di alto livello in un modello lineare. Si introducono pertanto i concetti di teoria dei compilatori che verranno applicati nel seguito: analisi lessicale e sintattica, grammatiche formali e *Parsing Expression Grammar* (PEG), albero di sintassi astratta (AST), analisi semantica e *type checking*, rappresentazione intermedia (IR) e *lowering* verso una forma più vicina alla macchina di destinazione (in questo caso, il solver).

Presentare le fasi classiche di un compilatore e mapparle esplicitamente sui capitoli successivi: parsing (Cap. 5), analisi semantica (Cap. 6), lowering/linearizzazione (Cap. 7), *back-end*/risoluzione (Cap. 8).

2.5 Posizionamento di ROOC

Sintetizzare in cosa ROOC si differenzia dagli strumenti esistenti: notazione vicina alla matematica, tipi ricchi (grafi, tuple, iterabili), estensibilità, esecuzione in WASM e finalità didattica.

Chapter 3

Il linguaggio ROOC

Capitolo di riferimento del linguaggio, dal punto di vista dell'utente: cosa si può scrivere e qual è il significato dei costrutti. L'implementazione è rimandata ai capitoli successivi.

3.1 Principi di design

Illustrare gli obiettivi di progetto: modello formale, leggibilità, vicinanza alla notazione matematica, separazione tra modello e dati.

3.2 Struttura di un modello

Un modello ROOC è composto dalla funzione obiettivo ($\min/\max$), dai vincoli ($s.t.$) e da due blocchi opzionali: `where` per la definizione di costanti e `define` per la dichiarazione del dominio delle variabili.

Mostrare la struttura generale con uno snippet commentato. Spiegare obiettivo, vincoli con iterazione (`for`), blocco `where` e blocco `define`.

3.3 Esempio guida

Presentare per esteso uno o due esempi (knapsack binario e dominating set su grafo) che verranno richiamati nei capitoli successivi per illustrare le varie fasi di compilazione.

3.4 Primitive e tipi di dato

Il linguaggio offre un insieme ricco di tipi: numeri, stringhe, booleani, *grafi*, tuple e iterabili. Si descrive la semantica di ciascuno.

Descrivere le primitive (Number, Integer, String, Boolean, Graph, Tuple, Iterable) e la loro rappresentazione a livello di tipo. Riferimento: [src/primitives/](#).

3.5 Iteratori, *block functions* e destructuring

Spiegare gli iteratori (`for ... in ...`), le *block scoped functions* (`sum`, `prod`, `min`, `max`, `avg`) e il destructuring di tuple e archi di grafo.

3.6 Operatori e definizione delle variabili

Trattare gli operatori e l'*operator overloading* sulle primitive, e la dichiarazione del dominio delle variabili con i relativi *bounds* (Boolean, Real, NonNegative-Real, IntegerRange).

3.7 Libreria standard e funzioni definite dall'utente

Elencare le funzioni built-in (su array, grafi, range) e descrivere il meccanismo di estensione con funzioni definite dall'utente (anche via JavaScript in ambiente WASM). Riferimento: [src/runtime_builtin/](#).

Chapter 4

Architettura del compilatore

Capitolo-ponte, breve: offre la visione d'insieme della pipeline che i capitoli successivi approfondiscono fase per fase.

4.1 La pipeline multi-fase

La compilazione di un modello ROOC procede attraverso una sequenza di trasformazioni, ciascuna delle quali raffina la rappresentazione del problema: dal codice sorgente al `PreModel`, al `Model` tipato, al `LinearModel`, alla forma standard e infine al *tableau* del simplesso.

Inserire un diagramma della pipeline completa (sorgente $\rightarrow$ `PreModel` $\rightarrow$ `Model` $\rightarrow$ `LinearModel` $\rightarrow$ `StandardLinearModel` $\rightarrow$ `Tableau` $\rightarrow$ soluzione). Descrivere input/output di ogni stadio.

4.2 L'astrazione *Pipe*

Ogni stadio della pipeline è modellato come una *Pipe* componibile, che trasforma un dato intermedio nel successivo. I dati intermedi sono entità di prima classe e serializzabili.

Descrivere il trait `Pipeable` e il `PipeRunner`. Spiegare come la composizione di pipe permetta di fermarsi a uno stadio qualsiasi. Riferimento: `src/pipe/`.

4.3 Estensibilità e visualizzazione passo-passo

Argomentare come l'architettura a pipe abiliti (1) l'estensibilità (nuovi solver/stadi) e (2) la visualizzazione passo-passo usata a fini didattici nella piattaforma web (Cap. 9).

Chapter 5

Analisi sintattica e rappresentazione intermedia

Primo capitolo del *front-end* del compilatore. Struttura del capitolo: prima i concetti teorici, poi la loro applicazione concreta in ROOC.

5.1 Concetti teorici

Si introducono i fondamenti dell'analisi lessicale e sintattica: il ruolo della grammatica formale nella definizione di un linguaggio, le *Parsing Expression Grammar* (PEG) come alternativa alle grammatiche context-free, e la costruzione di un albero di sintassi astratta (AST) a partire dal flusso di token.

TEORIA: definire grammatiche formali, PEG e differenze con CFG/BNF, concetto di AST e di rappresentazione intermedia. Spiegare perché si è scelto un parser PEG.

5.2 Applicazione in ROOC: la grammatica PEST

ROOC definisce la propria sintassi tramite una grammatica PEG espressa con la libreria PEST. Si analizzano le regole principali: obiettivo, vincoli, blocchi *where* e *define*, espressioni, iteratori.

APPLICAZIONE: descrivere `src/parser/grammar.pest` e il processo di parsing. Mostrare alcune regole significative e come gestiscono costrutti come la moltiplicazione implicita o le *block functions*.

5.3 Il PreModel e l'Intermediate Language

Il risultato del parsing è il PreModel, una rappresentazione intermedia (IL) che ricalca la struttura sintattica del sorgente prima dell'analisi semantica.

APPLICAZIONE: descrivere `PreExp`, `PreConstraint`, `PreObjective` e la costruzione del PreModel. Riferimento: `src/parser/pre_model.rs`, `src/parser/il/`.

5.4 Gestione e tracciamento degli errori

Descrivere il sistema di *span* e di *error trace* che collega ogni errore alla porzione di sorgente corrispondente. Riferimento: `src/utils.rs`.

Chapter 6

Analisi semantica: tipi e trasformazione del modello

Capitolo sull'analisi semantica. Anche qui: prima i concetti teorici, poi l'applicazione in ROOC.

6.1 Concetti teorici

Si presentano i concetti di analisi semantica nei compilatori: il *sistema dei tipi*, la verifica statica dei tipi (*type checking*), la nozione di *scope* e di tabella dei simboli, e la distinzione tra rappresentazione di un valore a tempo di esecuzione e la sua descrizione a livello di tipo.

TEORIA: cosa sono i sistemi di tipi, type checking statico vs dinamico, scope e ambienti (frame), inferenza/verifica delle firme di funzione.

6.2 Applicazione in ROOC: il sistema dei tipi

Il sistema dei tipi di ROOC è rappresentato da `PrimitiveKind`, che descrive a livello statico le primitive del linguaggio (inclusi tipi composti come iterabili e tuple) e governa le regole di applicabilità di operatori e funzioni.

APPLICAZIONE: descrivere `PrimitiveKind`, le regole degli operatori (`can_apply_binary_op`) e le firme delle funzioni. Riferimento: `src/primitives/primitive.rs`.

6.3 Il type checker

APPLICAZIONE: descrivere `TypeCheckerContext`, l'uso dei `Frame` per gli scope annidati (uno per livello di `for`), la mappa dei token tipati e la diagnostica dei tipi. Riferimento: `src/type_checker/`.

6.4 Trasformazione `PreModel` $\rightarrow$ `Model`

Superata la verifica dei tipi, il `PreModel` viene trasformato in un `Model`: le costanti vengono valutate, le variabili composte e gli iteratori risolti, e le *block functions* espanse in espressioni concrete.

APPLICAZIONE: descrivere `transform_parsed_problem`, il `TransformerContext`, il `recursive_set_resolver` per la risoluzione di iteratori e variabili composte, e l'espansione delle *block functions*. Riferimento: `src/parser/model_transformer/`.

Chapter 7

Linearizzazione e forma standard

Back-end del compilatore: traduzione del modello tipato in una forma lineare canonica adatta ai solver. Anche qui: prima la teoria, poi l'applicazione.

7.1 Concetti teorici

Si richiamano i concetti di linearità di un'espressione e le riformulazioni standard che consentono di esprimere costrutti non lineari come $\min$, $\max$ e valore assoluto tramite variabili ausiliarie e vincoli aggiuntivi. Si introduce inoltre la *forma standard* di un problema di programmazione lineare, prerequisito per il metodo del simplesso.

TEORIA: definire la linearità, le tecniche di linearizzazione di $\min/\max/\text{abs}$ con variabili ausiliarie, e la forma standard (vincoli di uguaglianza, variabili non negative, obiettivo di minimizzazione).

7.2 Applicazione in ROOC: da Model a LinearModel

Il `Model` può contenere costrutti non lineari; il *linearizer* attraversa l'albero delle espressioni, verifica la linearità e produce un `LinearModel` garantito lineare.

APPLICAZIONE: descrivere il `Linearizer`, il rilevamento della non-linearità (es. prodotto di variabili) e la struttura del `LinearModel`. Riferimento: `src/transformers/linearizer.rs`.

7.3 Linearizzazione di min, max e valore assoluto

APPLICAZIONE: mostrare con esempi come min/max vengano sostituiti da una variabile ausiliaria e dai relativi vincoli. Discutere lo stato della linearizzazione del valore assoluto (limite noto, cfr. conclusioni).

7.4 Trasformazione in forma standard

Il `LinearModel` viene infine normalizzato in forma standard: si introducono variabili di *slack* e *surplus*, si effettua lo *splitting* delle variabili libere e si normalizza la funzione obiettivo.

APPLICAZIONE: descrivere `to_standard_form`, l'aggiunta di slack/surplus, lo splitting delle variabili libere ($x = x' - x''$) e la normalizzazione dell'obiettivo. Riferimento: `src/transformers/standardizer.rs`.

Chapter 8

Risoluzione dei modelli

Capitolo focalizzato ma compatto sui solver. In primo piano il contributo originale microlp; i backend esterni sono trattati come opzioni intercambiabili. Mantenere la struttura teoria → applicazione.

8.1 Concetti teorici

Si richiamano gli algoritmi alla base della risoluzione: il metodo del simplesso e la sua variante a due fasi con variabili artificiali per la ricerca di una base ammissibile, e il *branch-and-bound* per l'estensione ai problemi con variabili intere e binarie (MILP).

TEORIA: simplesso (tableau, pivot, test del rapporto minimo), metodo a due fasi, branch-and-bound per MILP. Mantenere il livello adeguato a una tesi triennale.

8.2 Applicazione in ROOC: il simplesso a due fasi

ROOC include un'implementazione propria del simplesso a due fasi, capace di esporre i *tableau* intermedi per consentire la visualizzazione passo-passo.

APPLICAZIONE: descrivere l'implementazione del Tableau, le due fasi, l'estrazione della soluzione (`OptimalTableau`) e la registrazione degli step. Riferimento: `src/solvers/simplex/`.

8.3 **microlp**: fork ed estensione a MILP

Il solver **microlp** è un *fork* di un crate archiviato, originariamente limitato al solo semplice (variabili continue). Nell'ambito di questo lavoro è stato ripreso e mantenuto, ed esteso al supporto di variabili *interi* e *binarie*, diventando di fatto un solver MILP.

APPLICAZIONE/CONTRIBUTO: spiegare lo stato di partenza del fork (simplex continuo), le modifiche apportate per il supporto a variabili intere/binarie (branch-and-bound) e l'integrazione in ROOC. Questo è uno dei contributi originali della tesi.

8.4 Backend esterni e *auto-solver*

Oltre al solver proprio, ROOC può delegare la risoluzione a backend esterni in base al dominio delle variabili, tramite un meccanismo di *auto-solver*.

APPLICAZIONE: descrivere i backend (*good_lp*/Clarabel, *copper* per problemi binari, HiGHS nel browser) e la logica di *auto_solver* che seleziona il solver adatto. Riferimenti: `src/solvers/`.

8.5 Confronto qualitativo

Confronto qualitativo tra i solver disponibili (espressività, prestazioni, uso didattico). Eventuali piccoli esempi/benchmark.

Chapter 9

Dalla libreria alla piattaforma web

Capitolo di supporto: mostra come il motore (libreria Rust) diventi una libreria riusabile e una piattaforma web didattica. Mantenere il taglio sintetico, coerente con il focus sul motore.

9.1 Compilazione in WebAssembly e la libreria TypeScript

La libreria Rust viene compilata in WebAssembly tramite `wasm-pack` ed esposta a JavaScript/TypeScript attraverso `serde-wasm-bindgen`. Sopra di essa è costruita la libreria `@specy/rooc`, che offre un'API type-safe.

Descrivere il processo di build (Rust $\rightarrow$ WASM $\rightarrow$ pacchetto npm), la marshalling dei dati e l'API pubblica di `ts-lib`. Riferimenti: `src/lib.rs`, `src/pipe/pipe_wasm_runner.rs`, `ts-lib/`.

9.2 L'IDE web e l'LSP nel browser

La piattaforma web è un'applicazione SvelteKit con editor Monaco. Un Language Server Protocol (LSP) parziale, eseguito interamente nel browser, fornisce evidenziazione della sintassi, hover dei tipi, completamento ed evidenziazione degli

errori.

Descrivere l'IDE: editor Monaco con linguaggio ROOC custom, l'LSP nel browser (diagnostica, hover, completamento, formattazione), il rendering LaTeX con KaTeX e la persistenza locale (IndexedDB). Riferimenti: `frontend/src/lib/Monaco.ts`, `frontend/src/lib/Rooc/`.

9.3 Valore didattico

Argomentare il valore didattico: visualizzazione passo-passo della risoluzione, esecuzione di funzioni utente in sandbox, documentazione interattiva e condivisione dei modelli. Collegare all'astrazione a pipe (Cap. 4).

Chapter 10

Conclusioni e sviluppi futuri

10.1 Riepilogo dei contributi

Riepilogare i contributi della tesi: il linguaggio ROOC, il compilatore multi-fase, il solver MILP microlp e la piattaforma web didattica.

10.2 Limitazioni

Discutere le limitazioni note: linearizzazione del valore assoluto, assenza di vincoli logici, prestazioni del solver proprio rispetto ai backend industriali.

10.3 Sviluppi futuri

Delineare i possibili sviluppi: vincoli logici, ulteriori solver, miglioramenti del linguaggio e della piattaforma.

Appendix A

Grammatica completa di ROOC

Riportare la grammatica PEST completa (`src/parser/grammar.pest`), eventualmente commentata.

Appendix B

Esempi di modelli

Raccogliere esempi di modelli ROOC completi (knapsack, dominating set, ecc.) con relativo modello lineare compilato e soluzione.

Bibliography